DevOps Lab Cheat Sheet

A-1

Commands to demonstrate creating a project in local and remote repository using Git Bash and GitHub, applying `init`, `status`, `log`, `add`, `commit`, `push`, `config`, `clone`, and `reset` in Git`.

First time pushing code into a new repository

Step:1: git config --global user.name "YourName"

git config --global user.email "your_email@example.com"

Step 2: git init

Step 3: git status

Step 4: git add .

Step 5: git commit -m "Initial commit"

Step 6: git log

Step 7: git remote add origin https://github.com/username/repository-name.git

Step 8: git push -u origin master

Step 9: git clone https://github.com/username/repository-name.git

Step 10: git reset --soft HEAD~1

git reset --hard HEAD~1

If the repository already Exits

Step 1: git config --global user.name "YourName"

git config --global user.email your_email@example.com

Step 2: git clone https://github.com/username/repository-name.git

Step 3: cd repository-name

Step 4: git status

Step 5: git add .

Step 6: git commit -m "Updated commit"

Step 7: git log

Step 8: git push -u origin master

Step 9: git reset --soft HEAD~1

Step 10: git reset --hard HEAD~1

DevOps Lab Cheat Sheet

Erros	Solution
remote origin already exists	git remote remove origin git remote add origin https://github.com/username/repo.git
Force Update Remote Repository Account	git remote set-url origin https://username@gitlab.com/username/project.git

A-2

Demonstrate how to create a project in a remote repository and apply fork, merge, diff, merge conflict, branch, and pull request concepts on the repository using GitHub.

Step1: Create Repository in GitHub

Open **GitHub**.
Click **New Repository**.
Enter repository name: **git-demo**

Step 2: Clone Repository to Local System

Open terminal and run:
git clone https://github.com/username/git-demo.git
cd git-demo

Step 3: Create a file and Write the content

Hello GitHub

Step 4: Commit and Push

git add file1.txt
git commit -m "Initial commit"
git push origin main

Step 5: Create and Switch to Branch

git branch feature1
git checkout feature1

Step 6: Modify File in Feature Branch

Open file1.txt and add a line
Now Commit changes
git add file1.txt
git commit -m "Feature branch update"
git push origin feature1

DevOps Lab Cheat Sheet

Step 7: Show Difference Between Branches

```
git diff main feature1
```

Step 8: Create Merge Conflict

```
git checkout main
Open file1.txt and add another line
git add file1.txt
git commit -m "Main branch change"
git merge feature1
```

Step 9: Resolve Conflict

```
Open file1.txt and add another line
git add file1.txt
git commit -m "Merge conflict resolved"
git push origin main
```

Step 10: Fork and Create Pull Request

Open repository in GitHub.
Click Fork.
Modify the file in forked repository.
Click Pull Request.
Click Create Pull Request to request merge into original repository.

A-3

Demonstrate the process of integrating a GitHub repository with Jenkins to automate project execution in a CI/CD pipeline.

Requisites: GitHub Repository with Java/Python files
Jenkins(Go to https://www.jenkins.io, Go to https://www.jenkins.io)

To Unlock Jenkins:

Open a browser and go to: http://localhost:8080
Jenkins will ask for the Administrator password.
Copy the password from: C:\Program data\Jenkins\secrets\initialAdminPassword
Install Suggested Plugins

I. Manual Build creation

In Jenkins:

Click New Item
Enter project name
Select Freestyle Project
Click OK

DevOps Lab Cheat Sheet

Configure GitHub Repository

In Source Code Management

Select Git

Enter repository URL

Add Build Step

Scroll to Build → Add Build Step

Execute Windows Batch Command

Give the command (echo, python, java)

Save the Job

Run Build Now

II. Scheduled Build or Periodic Build (using Poll SCM)

In Jenkins:

Click New Item

Enter project name

Select Freestyle Project

Click OK

Configure GitHub Repository

In Source Code Management

Select Git

Enter repository URL

Enable Scheduled Build

Scroll to Build Trigger

Select Poll SCM

Mention * * * * *

Scroll to Build

Add Build Step → Execute Windows Batch Command

Give the command (echo, python, java)

Save

Make a change in the GitHub repository and the code.

Jenkins will detect the change using Poll SCM and automatically start the build.

III. Event-Based Automatic Build using GitHub Webhooks

In Jenkins:

Click New Item

Enter project name

Select Freestyle Project

Click OK

Configure GitHub Repository

Scroll to **Source Code Management**

Select **Git**

Enter the GitHub repository URL

Scroll to Build Triggers

Select: GitHub hook trigger for GITScm polling

Install and Run ngrok

ngrok http 8080 (Jenkins port)

Add Build Step

Scroll to Build → Add Build Step
Execute Windows Batch Command
Give the command (echo, python, java)
Save the Job

Configure Webhook in GitHub

Open your repository in GitHub
Go to Settings
Click Webhooks
Click Add Webhook
Enter Webhook Details:
Payload URL: http://ngrok_generated_IP/**github-webhook/**
Click Add Webhook
Make a change in the project and push the code

When code is pushed to GitHub, the Webhook sends a request through ngrok to Jenkins, and Jenkins immediately starts the build.

B1

Create a Docker image for an application stored in a local repository and run the application using docker image.

Step1: Download Docker Desktop from the official website:

https://www.docker.com/products/docker-desktop
Install Docker Desktop.
Start Docker Desktop.
Verify installation in terminal / command prompt**: docker –version**

Step 2: Create a Local Application

Create a folder for the application**. Eg*: myapp***
Inside the folder create a simple Python application. ***App.py***

Step 3: Create a Dockerfile

Inside the same folder create a file named**: Dockerfile(**without .txt extension)
Add the following content:
FROM python:3.10
WORKDIR /app
COPY . .
CMD ["python","app.py"]

Step 4: Open terminal in the myapp folder.

docker build -t myapp .

Step 5: Verify the docker image

docker images

Step 6: Run the Docker Container

docker run myapp

DevOps Lab Cheat Sheet

B2: Create and configure Jenkins files for workflow and build of an application and push the image on docker hub..

Step 1: Build Docker Image from Your Folder
cd your-folder-name
Build the image: ***docker build -t imagename*** .

Step 2: Add a Tag:
docker tag imagename yourdockerhubusername/imagename:tagname
eg: *docker tag myapp akshatha123/myapp:v1*

Step 3: Login to Docker Hub

Step 4: Push Image to Docker Hub

docker push yourdockerhubusername/imagename:tagname
eg: *docker push akshatha123/myapp:v1*

Step 5: Verify on Docker Hub

Step 6: Push the code to Github
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/your-username/docker_demo.git
git push -u origin main

Step 7: Add Docker Hub Credentials in Jenkins
Open Jenkins
Manage Jenkins → Credentials → Configure Credentials

Add:

Kind: Username & Password
ID: `dockerhub-creds`
Username: your Docker Hub username
Password: **Docker Hub Access Token**

How to Check Docker Access Token

DevOps Lab Cheat Sheet

Step 1: Open Security Settings

Login to Docker Hub

Click your profile (top right)

Go to:

`Account Settings → Security`

Open **Access Tokens**

In command prompt:

```
echo YOUR_TOKEN | docker login -u your-username --password-stdin
```

Step 8: Create Jenkins Pipeline Job

Jenkins Dashboard → **New Item**
Select **Pipeline**
Configure:
Pipeline → *Pipeline script from SCM*
SCM → Git
Repo URL → your GitHub repo
Script Path → `Jenkinsfile`

Step 9: Build Now (manual)

Docker Automation using Webhook

Start by creating a new empty repository

Step 1: Required Background Services (One-Time Setup)

Jenkins

http://localhost:8080

Docker Desktop

Must be **running in background**

ngrok

```
ngrok http 8080
```

Step 2: Set up the Webhook

Step 3: Jenkins Job Config

GitHub hook trigger for GITScm polling

Step 4: Push the code to GitHub

Jenkins should automatically build, else check the plugins

DevOps Lab Cheat Sheet

C1. Create a maven projects with all dependencies required for the application in CI/CD pipeline.

To run CI/CD pipeline in GitHub

I. Install required tools

II. **Install Java (JDK 11 or higher)**
- Download from Oracle JDK or OpenJDK
- Set environment variable:
  - JAVA_HOME = C:\Program Files\Java\jdk-XX
  - Add %JAVA_HOME%\bin to PATH

III. **Install Apache Maven**
- Download and extract
- Set:
  - MAVEN_HOME = C:\apache-maven
  - Add %MAVEN_HOME%\bin to PATH

IV. **Install Visual Studio Code**

V. Install VS Code Extensions:
- **Extension Pack for Java**
- **Maven for Java**
- **Debugger for Java**

II. Create Maven Project in VS Code

Open VS Code
Press Ctrl + Shift + P
Type: **Java: Create Java Project**
Select Maven create from archetype
Choose:
Archetype → maven-archetype-quickstart
Enter details:
Group Id: com.example
Artifact Id: *demo* (Give any name)
Choose folder → **Project will be created automatically**

Push the folder to git repository

III. For CI/CD Pipeline (GitHub Actions)

In the local folder
Create a folder by name **.github** and inside that create **worflows folder and in that create a file maven.yml**
Structure:
Localfoldername
-demo
-.github/workflows/maven.yml
Push this to Github repo

IV. Check the actions tab in github for the CI working

DevOps Lab Cheat Sheet

maven.yml

```
name: Java CI/CD Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build:
    runs-on: windows-latest
    env:
      FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true

   steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        distribution: 'temurin'
        java-version: '11'

    - name: Build using Maven
      run: mvn clean install
      working-directory: demo
```

C2:Integrate communication channel with Jenkins for status of project and also enable email notification for a build.

Update the plugins in Jenkins
Git Plugin
Pipeline Plugin
Maven Integration Plugin

Create a **Jenkinsfile** of type file and push it to the repo

Jenkinsfile

```
pipeline {
   agent any

   tools {

       maven 'Maven3'
   }

   stages {
```

```
    stage('CHECKOUT') {
      steps {
        git 'your_github_repo_link'
      }
    }

    stage('Build') {
      steps {
        dir('demo'){
        bat 'mvn clean install'
      }
      }
    }
    stage('Test') {
      steps {
        dir('demo'){
        bat 'mvn test'
      }
      }
    }
  }
}
```

In Jenkins goto Manage Jenkins
Tools
Goto Maven installations
Click on Maven installations
Add maven
Name= Maven3
Uncheck auto installations
Maven_Home path C:\Program Files\maven
Save
Now Click on Build Now

For Notification – Slack Tool Usage

Login to slack.com
Create a workspace
Ex: with any name given – it looks like **hello-odm9638**
Login to ***https://my.slack.com/services/new/jenkins-ci***
Select the hello workspace
Here token will be generated **copy that token**

Now in Jenkins

New item- freestyle project- ok
In configure- build steps- windows batch command- echo Jenkins
In post-build-action select **Slack notification**
And check all the options
Save

DevOps Lab Cheat Sheet

Enable Slack notification in Jenkins

Managejenkins- System
Slack
Workspace= **hello-odm9638**
Credential= add – secret text
Secret= **(copied token)**
ID= any name ex: slack-token
Channel= (channel name from slack website)
ok